



UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO

MAESTRÍA EN INTELIGENCIA ARTIFICIAL

ASIGNATURA:

PROYECTO INTEGRADOR EN INTELIGENCIA ARTIFICIAL

TÍTULO:

**ANÁLISIS COMPARATIVO DE TÉCNICAS DE INTELIGENCIA ARTIFICIAL
PARA TEXT-TO-SQL CON VISUALIZACIÓN AUTOMÁTICA**

AUTORES:

**SÁNCHEZ ARÉVALO ANÍBAL ANDRÉS
CABRERA PARRALES JOEL DAVID**

TUTOR:

Mg. ALEXANDRA ARCINIEGAS C.

SAMBORONDÓN, ECUADOR

ABRIL 2025

Introducción

Entre las distintas familias de modelos de inteligencia artificial evaluadas, los Transformers y los modelos de lenguaje grande (LLM) de código abierto constituyen el enfoque más efectivo para construir sistemas Text-to-SQL orientados a usuarios sin conocimiento técnico en SQL. Esta conclusión se sustenta en evidencia empírica sólida: los sistemas basados en esta arquitectura alcanzan hasta el 86.6 % de precisión de ejecución en el benchmark Spider con estrategias avanzadas de prompting, y hasta el 79.9 % con modelos de código abierto ajustados mediante fine-tuning como LLaMA 3. Estos resultados emergen de un análisis sistemático de seis familias técnicas de inteligencia artificial, evaluadas en función de las dimensiones críticas del problema: generación estructurada de SQL a partir de lenguaje natural en español, seguridad en la ejecución de consultas, capacidad de generalización a dominios no vistos durante el entrenamiento, y producción de visualizaciones automáticas a partir de los datos recuperados.

El problema Text-to-SQL representa una instancia del parsing semántico en la que una consulta formulada en lenguaje natural debe ser transformada en una instrucción SQL ejecutable sobre una base de datos relacional. La dificultad del problema radica en tres desafíos interrelacionados: el schema linking —proceso de vincular las entidades léxicas de la pregunta con los elementos del esquema de la base de datos—, la composicionalidad del SQL —que abarca operaciones de JOIN, subconsultas anidadas y funciones de agregación—, y la generalización a esquemas no observados durante el entrenamiento (Yu et al., 2018). El presente análisis evalúa cada técnica desde su fundamento teórico, su desempeño en benchmarks de referencia, su complejidad computacional, sus requerimientos de datos y su adecuación al contexto del proyecto: un sistema de código abierto que recibe preguntas en español, genera SQL con restricciones de seguridad, consulta bases de datos relacionales y devuelve los resultados acompañados de visualizaciones automáticas para perfiles de gerencia y operaciones.

1. Transformers y Modelos de Lenguaje Grande: La Arquitectura Dominante

Fundamentación Teórica

La arquitectura Transformer, propuesta originalmente por Vaswani et al. (2017), opera a través de mecanismos de autoatención (self-attention) capaces de capturar relaciones entre todos los pares de tokens de una secuencia de manera simultánea. Esta capacidad elimina la limitación de procesamiento secuencial propia de las redes recurrentes. Para el problema Text-to-SQL se emplean dos paradigmas principales: modelos encoder-decoder como T5 (Raffel et al., 2020), en los que el encoder procesa la concatenación de la pregunta en lenguaje natural con el esquema serializado de la base de datos y el decoder genera autorregresivamente la consulta SQL; y modelos decoder-only como LLaMA (Touvron et al., 2023) y CodeLlama (Rozière et al., 2023), que abordan la tarea mediante aprendizaje en contexto (in-context learning) o ajuste fino por instrucción.

La autoatención en el encoder desempeña un papel fundamental en el schema linking, dado que captura las relaciones existentes entre los tokens de la pregunta y los elementos del esquema —nombres de tablas, columnas, tipos de datos y claves foráneas—. La atención cruzada en el decoder permite referenciar de manera simultánea tanto la pregunta como el esquema durante la generación de cada token SQL. Entre las adaptaciones específicas para esta tarea destaca PICARD (Scholak et al., 2021), que incorpora decodificación restringida incremental para verificar la validez gramatical del SQL generado en cada paso, reduciendo los errores de ejecución del 12 % al 2 %. RESDSQL (Li et al., 2023a) separa el proceso de

schema linking del parsing de esqueleto SQL mediante un cross-encoder que filtra previamente los elementos del esquema más relevantes. Graphix-T5 (Li et al., 2023b), por su parte, incorpora capas de atención relacional sobre grafos dentro de T5 para codificar la estructura relacional entre tablas y columnas.

Para los modelos decoder-only, el paradigma de prompting con descomposición ha demostrado resultados superiores. DIN-SQL (Pourreza & Rafiei, 2023) divide el problema en cuatro submódulos secuenciales: schema linking, clasificación y descomposición de la consulta, generación del SQL y autocorrección. DAIL-SQL (Gao et al., 2024) optimiza sistemáticamente las estrategias de ingeniería de prompts, incluyendo representación de preguntas, selección de ejemplos basada en similitud de esqueleto SQL y organización de las demostraciones. La técnica de Generación Aumentada por Recuperación (RAG) complementa estos enfoques embebiendo descripciones de tablas y columnas en almacenes vectoriales para recuperar los elementos de esquema semánticamente más pertinentes para cada pregunta, lo que resulta especialmente valioso en bases de datos empresariales cuyos esquemas exceden la ventana de contexto del modelo.

Desempeño en Benchmarks

Los sistemas basados en Transformers dominan de manera consistente todos los benchmarks contemporáneos de Text-to-SQL. En Spider (Yu et al., 2018), el sistema MiniSeek alcanzó el 91.2 % de precisión de ejecución (EX) en el conjunto de prueba, siendo el primero en superar el umbral del 90 %. DAIL-SQL con GPT-4 obtuvo el 86.6 % EX con votación por autoconsistencia (Gao et al., 2024). DIN-SQL con GPT-4 logró el 85.3 % EX (Pourreza & Rafiei, 2023), mientras que C3 con ChatGPT alcanzó el 82.3 % EX en modo zero-shot, empleando aproximadamente 1,000 tokens por consulta (Dong et al., 2023).

En el segmento de código abierto los resultados son igualmente destacados. RESDSQL-3B con NatSQL logró el 84.1 % EX en desarrollo y el 79.9 % EX en prueba de Spider (Li et al., 2023a). LLaMA 3 8B Instruct ajustado mediante fine-tuning alcanzó el 79.9 % EX en desarrollo, superando el rendimiento zero-shot de GPT-4 (72.3 %). CodeLlama-13B con ajuste LoRA obtuvo el 78.9 % EX en desarrollo, entrenado en aproximadamente doce horas sobre una GPU A100. T5-3B con PICARD logró el 79.3 % EX en desarrollo y el 75.1 % EX en prueba (Scholak et al., 2021). En el benchmark BIRD (Li et al., 2024a), que incorpora datos ruidosos y conocimiento externo, DIN-SQL con GPT-4 alcanzó el 55.9 % EX en prueba, mientras que CodeS-7B (Li et al., 2024b) obtuvo el 57.0 % EX en desarrollo, frente a una precisión humana experta del 92.96 %.

Complejidad Computacional y Requerimientos

Modelo	Parámetros	VRAM Inferencia	VRAM Entrenamiento (QLoRA)	Tiempo Fine-Tuning
T5-Base	220 M	≈ 0.5 GB	≈ 1.7 GB	Horas (1 GPU)
T5-3B	2.9 B	≈ 6.4 GB	≈ 21 GB (FP16)	≈ 24 h (A100 80 GB)

POSTGRADO MAESTRÍA EN INTELIGENCIA ARTIFICIAL	“PROYECTO INTEGRADOR EN INTELIGENCIA ARTIFICIAL”	
---	---	--

CodeLlama-7B	7 B	≈ 14 GB	≈ 10 GB (4-bit)	≈ 4 h (2× V100)
CodeLlama-13B	13 B	≈ 28 GB	≈ 16 GB (4-bit)	≈ 12 h (A100 40 GB)
LLaMA-3-8B	8 B	≈ 17 GB	≈ 12 GB (4-bit)	Pocas horas (1 GPU)

El ajuste fino sobre el conjunto de entrenamiento de Spider (aproximadamente 8,659 ejemplos) produce resultados competitivos. Las técnicas LoRA y QLoRA (cuantización a 4 bits) reducen sustancialmente los requerimientos de memoria, permitiendo el ajuste de modelos de 13 B parámetros en hardware accesible. La inferencia con T5-3B optimizado mediante TensorRT sobre GPU A100 logra una aceleración de 21 veces respecto a la inferencia en CPU con PyTorch. Los modelos de código abierto como CodeS (1 B / 3 B / 7 B), disponibles bajo licencias permisivas y diseñados específicamente para Text-to-SQL, ofrecen un balance óptimo entre rendimiento y eficiencia computacional.

Ventajas y Limitaciones

Las principales ventajas de esta familia técnica incluyen: capacidad para generar SQL complejo de manera extremo a extremo —con JOINS, subconsultas y funciones de agregación—; aprovechamiento del conocimiento masivo adquirido durante el preentrenamiento en código y lenguaje natural; soporte multilingüe incorporado, con una degradación de apenas el 6.1 % para el español respecto al inglés según MultiSpider (Dou et al., 2023); y un ecosistema de código abierto maduro con frameworks como DB-GPT-Hub y DAIL-SQL. Las limitaciones comprenden: latencia de inferencia superior a métodos más simples, especialmente con llamadas API múltiples en métodos de descomposición; riesgo de generar SQL sintácticamente correcto pero semánticamente incorrecto (alucinaciones); y degradación notable en escenarios empresariales reales, como evidencia Spider 2.0, donde el mejor agente (o1-preview) alcanza apenas el 21.3 %, en contraste con el 91.2 % obtenido en Spider 1.0 (Lei et al., 2024).

2. Redes Neuronales Recurrentes: Precursores del Parsing Semántico Neural

Fundamentación Teórica

Las redes LSTM (Long Short-Term Memory) y GRU (Gated Recurrent Unit) procesan secuencias de manera token a token, manteniendo un estado oculto que captura dependencias temporales mediante mecanismos de compuertas. En el paradigma Seq2Seq aplicado a Text-to-SQL, un encoder LSTM bidireccional codifica la pregunta en lenguaje natural produciendo una secuencia de estados ocultos, mientras un decoder LSTM genera autorregresivamente la consulta SQL. El mecanismo de atención de Bahdanau (2015) permite al decoder atender selectivamente a distintas posiciones de la entrada en cada paso de decodificación. Una extensión fundamental para Text-to-SQL son las redes de puntero (Pointer Networks) de Vinyals et al. (2015), que permiten al decoder copiar tokens directamente desde la secuencia de entrada —como nombres de columnas, tablas y valores literales— en lugar de generarlos desde un vocabulario fijo.

Desempeño y Evolución Histórica

Seq2SQL (Zhong et al., 2017) representó el primer enfoque de aprendizaje profundo a gran escala para Text-to-SQL, e introdujo simultáneamente el dataset WikiSQL con 80,654 pares pregunta-SQL. Logró el 59.4 % de precisión de ejecución en WikiSQL, superando en un 65 % relativo al baseline Seq2Seq atencional (35.9 %). SQLNet (Xu et al., 2017) elevó ese resultado a aproximadamente el 68 % EX en WikiSQL mediante un enfoque basado en sketch que evita la generación secuencial para componentes no ordenados del SQL. TypeSQL (Yu et al., 2018a) incorporó información de tipos semánticos, alcanzando el 82.6 % EX en WikiSQL con acceso al contenido de la tabla.

Sin embargo, estos modelos mostraron un colapso de rendimiento al transferirse al benchmark Spider de consultas multitabla y multi-dominio. Los baselines Seq2Seq originales alcanzaron apenas el 12.4 % de exact match en Spider, y SyntaxSQLNet llegó únicamente al 19.7 % (Yu et al., 2018b). Esta brecha de más de 60 puntos porcentuales expuso las limitaciones estructurales de las arquitecturas recurrentes para el SQL complejo.

Limitaciones Estructurales

Las redes recurrentes presentan restricciones intrínsecas para Text-to-SQL: degradación de información en secuencias largas a pesar de las compuertas LSTM; procesamiento estrictamente secuencial que impide la paralelización durante el entrenamiento; incapacidad para modelar las relaciones estructurales del esquema —claves foráneas, jerarquías tabla-columna— al tratarlo como secuencia plana; e imposibilidad de beneficiarse del preentrenamiento a gran escala de modelos como BERT o GPT. Su complejidad computacional es $O(T)$ por capa y su eficiencia en memoria —apenas 4 GB de VRAM para entrenamiento— no compensa la brecha de rendimiento de más de 60 puntos porcentuales respecto a los Transformers en benchmarks complejos.

3. Redes Neuronales con Grafos para la Codificación de Esquemas Relacionales

Fundamentación Teórica

Las Graph Neural Networks (GNN) aprovechan la estructura inherentemente grafular de los esquemas de bases de datos relacionales, en los que las tablas y columnas constituyen nodos y las relaciones —claves foráneas, pertenencia columna-tabla, claves primarias— forman aristas tipificadas. El mecanismo central es el paso de mensajes: cada nodo actualiza su representación agregando información de sus vecinos, con distintos tipos de aristas que pueden emplear matrices de pesos independientes. Tras L capas de paso de mensajes, la representación de cada nodo incorpora información de su vecindario de L saltos.

RAT-SQL (Wang et al., 2020) generaliza este enfoque mediante autoatención relacional construyendo un grafo completo sobre todos los tokens de la pregunta y los nodos de tabla y columna, con tipos de arista que codifican tanto relaciones estructurales como vínculos de schema linking. LGESQL (Cao et al., 2021) introduce una innovación topológica mediante la construcción de un grafo de línea a partir del grafo de esquema original, donde las aristas se convierten en nodos conectados cuando comparten un extremo, lo que permite propagar mensajes sobre la topología misma de las relaciones.

Desempeño en Spider

POSTGRADO MAESTRÍA EN INTELIGENCIA ARTIFICIAL	“PROYECTO INTEGRADOR EN INTELIGENCIA ARTIFICIAL”	
---	---	--

Sistema	Dev EM	Test EM	Encoder preentrenado
RAT-SQL (sin BERT)	57.2 %	—	GloVe
RAT-SQL + BERT	69.7 %	65.6 %	BERT-large
RAT-SQL + GraPPa	≈ 73.4 %	69.6 %	GraPPa (RoBERTa)
LGESQL + ELECTRA	72.0 %	—	ELECTRA
Graphix-T5 3B + PICARD	—	77.6 % (EX)	T5-3B

Los resultados revelan un patrón consistente: las GNN son más efectivas como componentes dentro de arquitecturas híbridas que como sistemas autónomos. RAT-SQL sin preentrenamiento obtuvo el 57.2 % EM, pero combinado con GraPPa escaló al 73.4 %. Graphix-T5 integra capas con sesgo inductivo de grafos directamente en T5, demostrando que este enfoque puede inyectarse en Transformers seq2seq. La complejidad computacional de las GNN es moderada: $O(V + E)$ por capa para implementaciones dispersas, con grafos de esquema típicos de menos de 100 nodos y 500 aristas. El componente GNN agrega sobrecarga marginal respecto al modelo de lenguaje subyacente, con tiempos de entrenamiento de doce a veinticuatro horas en una sola GPU para RAT-SQL.

4. Random Forest y Gradient Boosting como Clasificadores de Subtareas

Fundamentación Teórica

Random Forest (Breiman, 2001) construye un ensemble de árboles de decisión independientes mediante bagging y aleatorización de características, donde la clase predicha se determina por votación de mayoría. Gradient Boosting (Friedman, 2001), implementado en XGBoost (Chen & Guestrin, 2016) y LightGBM (Ke et al., 2017), construye árboles secuencialmente donde cada nuevo árbol corrige los errores residuales del ensemble previo mediante descenso de gradiente funcional. LightGBM emplea crecimiento orientado a hojas y optimización basada en histogramas, manejando nativamente matrices dispersas de TF-IDF, lo que lo hace especialmente adecuado para clasificación de texto.

Aplicación al Problema Text-to-SQL

Estos métodos no son aplicables a la generación extremo a extremo de SQL, dado que producen predicciones de clase fija y no secuencias de longitud variable con estructura composicional. Su utilidad se restringe a subtareas de clasificación dentro de pipelines de PLN más amplios: identificación del tipo de consulta (SELECT, COUNT, GROUP BY), selección de tablas y columnas relevantes, y predicción de cláusulas SQL. En el framework MindMeld (Cisco), Random Forest alcanzó el 90.50 % de precisión en clasificación de intención, mientras que SVM lineal logró el 97.41 % en la misma tarea.

Un hallazgo relevante de Bohacek y Bravansky (2024) muestra que los pipelines basados en XGBoost con ingeniería de características pueden superar a GPT-4 y LLaMA 2 en tareas específicas de clasificación de texto. No obstante, las limitaciones son fundamentales para Text-to-SQL: ausencia de capacidad generativa secuencial, imposibilidad de modelar composicionalidad SQL, espacio de salida

ilimitado e incapacidad para realizar schema linking a nivel de secuencia. La complejidad de entrenamiento es del orden de $O(n \times m \times T \times d)$, completándose generalmente en minutos a pocas horas sin necesidad de GPU.

5. Support Vector Machines para Clasificación de Intención en PLN

Fundamentación Teórica

Las SVM (Vapnik, 1995) identifican el hiperplano óptimo que maximiza el margen entre clases en un espacio de alta dimensionalidad, con los vectores de soporte como los puntos más próximos a la frontera de decisión. El kernel trick permite operar implícitamente en espacios de dimensionalidad superior: el kernel lineal resulta óptimo para representaciones textuales TF-IDF de alta dimensionalidad y dispersas, mientras que el kernel RBF captura patrones no lineales para clasificaciones más complejas. El parámetro de regularización C controla el equilibrio entre amplitud del margen y tolerancia a errores de clasificación.

Aplicación y Desempeño en Contextos PLN Relevantes

Las SVM fueron históricamente el clasificador dominante en tareas de PLN previo al aprendizaje profundo. Haffner et al. (2003) redujeron la tasa de error en clasificación de llamadas en un 50 % utilizando SVM con n-gramas en el sistema de AT&T. En el dataset ATIS con 4,978 ejemplos y 26 categorías de intención, los sistemas basados en SVM reportaron tasas de error inferiores al 5 % para clasificación de intención.

El estudio comparativo de Wahba et al. (2022) confrontó LinearSVC con TF-IDF frente a BERT, RoBERTa, XLNet y DistilBERT en cuatro conjuntos de datos, encontrando que SVM superó a todos los modelos de lenguaje preentrenados en el dataset 20NewsGroup ($F1 = 0.93$) e igualó a RoBERTa en Consumer Complaints ($F1 = 0.98$). No obstante, al igual que los métodos de ensemble, las SVM no pueden generar secuencias SQL: su aplicabilidad al problema Text-to-SQL se limita a componentes de clasificación dentro de pipelines más amplios, y su complejidad de entrenamiento $O(n^2)$ a $O(n^3)$ limita la escalabilidad con kernels no lineales en datasets grandes.

6. Sistemas Basados en Reglas y Gramáticas: El Enfoque Determinista

Fundamentación Teórica

Los sistemas basados en reglas emplean coincidencia de patrones, expresiones regulares, análisis de gramáticas libres de contexto (CFG) y parsing de dependencias para transformar lenguaje natural en SQL. La arquitectura típica comprende: un analizador léxico que tokeniza la pregunta; un parser sintáctico que genera árboles de dependencia mediante herramientas como spaCy o NLTK; un módulo de mapeo semántico que vincula los nodos del árbol con componentes SQL a través de reglas artesanales; y un generador SQL basado en plantillas que ensambla la consulta final.

Sistemas Históricos y Contemporáneos

LUNAR (Woods et al., 1972) fue uno de los primeros sistemas de interfaz en lenguaje natural para bases de datos, orientado a consultas geológicas sobre muestras lunares. NaLIR (Li & Jagadish, 2014), publicado en ACM SIGMOD, representa el estado del arte de los enfoques basados en reglas: emplea el Stanford NLP Parser para generar árboles de dependencia, un mapper de nodos que vincula elementos del parse tree con componentes SQL, y un ajustador estructural que infiere el significado semántico. Cuando

existe ambigüedad, NaLIR presenta múltiples interpretaciones al usuario para selección interactiva. Los sistemas heurísticos basados en plantillas han reportado hasta el 93 % de precisión en conjuntos controlados de más de 200 consultas, mapeando directamente patrones léxicos a operadores SQL.

Las ventajas de estos enfoques son claras: no requieren datos de entrenamiento, son deterministas y reproducibles, generan SQL sintácticamente válido por construcción y su inferencia es computacionalmente trivial (milisegundos). Sin embargo, sus limitaciones son igualmente definidas: fragilidad ante variaciones de fraseo, cobertura restringida al conjunto explícito de patrones, dificultad exponencial con consultas complejas, y elevado costo de desarrollo para cada idioma. Para el español, con su orden de palabras flexible, concordancia de género y número, y pronombres clíticos, la implementación se vuelve particularmente compleja. La encuesta NLI4DB (2025) señala que diseñar una regla o plantilla para cada escenario posible se ha vuelto cada vez más difícil e impráctico conforme crece la complejidad de las bases de datos.

7. Matriz Comparativa Teórica de las Seis Técnicas Evaluadas

Dimensión	Transf./LLM	RNN/LSTM	GNN	RF/GB	SVM	Reglas
Capacidad generativa SQL	End-to-end completa	End-to-end limitada	Componente de codificación	Solo clasificación	Solo clasificación	Plantillas predefinidas
Mejor EX en Spider	91.2 % (MiniSeek)	12-19 % EM	73.4 % EM (con PLM)	N/A	N/A	N/A
Complejidad SQL	Alta (JOINS, subconsultas)	Baja (SELECT-WHERE)	Media-Alta (componente)	N/A	N/A	Baja-Media
Datos de entrenamiento	0 (zero-shot) - 8 K	+ 10 K pares	+ 5 K pares + esquemas	+ 1 K etiquetados	+ 500 etiquetados	Sin datos
VRAM inferencia	0.5 - 28 GB	< 1 GB	< 2 GB	< 1 GB (CPU)	< 1 GB (CPU)	Mínima (CPU)
Latencia inferencia	0.5 - 5 seg	0.1 - 1 seg	0.05 - 0.5 seg	< 50 ms	< 10 ms	< 5 ms
Soporte multilingüe	Nativo (MultiSpider)	Limitado	Agnóstico al idioma	Requiere features	Requiere features	Requiere reescritura
Generalización cross-domain	Alta	Muy baja	Media-Alta (componente)	Baja	Baja	Muy baja
Interpretabilidad	Baja (caja negra)	Baja	Media (grafo explícito)	Alta	Media-Alta	Muy alta
Disponibilidad open source	Excelente	Buena (legacy)	Buena	Excelente	Excelente	Variable

8. Estado del Arte y Evaluación en Benchmarks de Referencia

Benchmarks Fundamentales

El benchmark Spider (Yu et al., 2018) constituye el estándar principal para la evaluación de Text-to-SQL en escenarios multi-dominio, con 10,181 preguntas y 5,693 consultas SQL únicas distribuidas en 200 bases de datos de 138 dominios. Spider evalúa cuatro niveles de dificultad e incluye JOINS, GROUP BY, ORDER BY, HAVING, subconsultas y operaciones de conjunto. Su tabla de clasificación fue cerrada en febrero de 2024 tras alcanzar el 91.2 % EX, y el campo ha migrado progresivamente hacia benchmarks más desafiantes.

BIRD (Li et al., 2024a), presentado en NeurIPS 2024, introduce 12,751 pares text-to-SQL sobre 95 bases de datos de 37 dominios profesionales, incorporando datos ruidosos, requerimientos de conocimiento externo y evaluación de eficiencia SQL. La brecha entre ChatGPT (40.08 % EX) y expertos humanos (92.96 %) ilustra la dificultad del benchmark. Spider 2.0 (Lei et al., 2024), presentado como oral en ICLR 2025, eleva la complejidad a escenarios empresariales reales con 632 problemas sobre bases con más de 1,000 columnas; el mejor agente alcanza apenas el 21.3 %, revelando la distancia entre los benchmarks académicos y los despliegues productivos.

Para el contexto multilingüe, MultiSpider (Dou et al., 2023) extiende Spider a siete idiomas incluyendo el español, documentando una degradación absoluta del 6.1 % en precisión para idiomas distintos al inglés y proponiendo el framework SAVE que cierra esa brecha en un 29.5 %.

Métricas de Evaluación

Tres métricas principales gobiernan la evaluación en el campo. Exact Match Accuracy (EM) verifica la coincidencia exacta entre la consulta predicha y la de referencia descomponiéndolas en conjuntos de componentes; tiende a subestimar el rendimiento al no reconocer consultas semánticamente equivalentes pero estructuralmente distintas. Execution Accuracy (EX) ejecuta ambas consultas sobre la base de datos real y compara los conjuntos de resultados; puede sobreestimar si consultas incorrectas producen resultados coincidentes en una instancia específica. Test-Suite Accuracy (TS), métrica oficial de Spider desde noviembre de 2020 (Zhong et al., 2020), mitiga ambas limitaciones ejecutando las consultas sobre colecciones diversas de instancias de base de datos generadas mediante fuzzing sistemático, con una correlación del 74-90 % respecto a EX tradicional.

9. Justificación Fundamentada para la Selección de Transformers y LLMs

La selección de Transformers y LLMs como técnica óptima para el proyecto específico —un sistema de código abierto que recibe preguntas en español de usuarios no técnicos, genera SQL con restricciones de seguridad sobre bases de datos relacionales y produce visualizaciones automáticas— se fundamenta en siete argumentos convergentes.

Primero, superioridad empírica demostrada. Los Transformers dominan todos los benchmarks contemporáneos con márgenes que hacen inviables las alternativas. En Spider, la brecha entre el mejor sistema Transformer (91.2 % EX) y el mejor modelo RNN (12–19 % EM) supera los 70 puntos porcentuales. Los modelos de código abierto ajustados (LLaMA-3-8B al 79.9 % EX, RESDSQL-3B al 84.1 % EX en desarrollo) superan ampliamente cualquier enfoque alternativo.

Segundo, capacidad generativa para SQL complejo. Únicamente los Transformers pueden generar extremo a extremo consultas SQL arbitrariamente complejas con JOINS múltiples, subconsultas correlacionadas, funciones de ventana y operaciones de conjunto. Las RNN fracasaron catastróficamente ante esta complejidad; las GNN operan como componentes de codificación; los métodos de clasificación no pueden producir secuencias; y los sistemas basados en reglas se limitan a plantillas predefinidas.

Tercero, soporte nativo para el español. Los LLMs multilingües preentrenados poseen conocimiento del español incorporado desde el preentrenamiento, y MultiSpider demuestra que la degradación es manejable ($\approx 6\%$). Los sistemas basados en reglas requerirían reescritura completa; las SVM y RF necesitarían ingeniería de características específica; las RNN carecen de representaciones multilingües transferibles.

Cuarto, flexibilidad de despliegue de código abierto. El ecosistema incluye modelos optimizados para Text-to-SQL (CodeS 1B/3B/7B), frameworks completamente abiertos (DB-GPT-Hub, DAIL-SQL) y técnicas de cuantización (QLoRA 4-bit) que permiten la inferencia de modelos de 7 B parámetros en GPUs con 10 GB de VRAM.

Quinto, compatibilidad con RAG para esquemas dinámicos. La arquitectura RAG permite gestionar bases de datos empresariales con esquemas extensos embebiendo las descripciones CREATE TABLE en almacenes vectoriales y recuperando los k elementos más relevantes para cada pregunta. Sistemas como DFIN-SQL y CSR-RAG demuestran la efectividad de este patrón en entornos productivos.

Sexto, integración natural con visualización automática. La investigación de Luo et al. (2021, SIGMOD) establece formalmente la conexión entre NL2SQL y NL2VIS. El stack Transformer unifica naturalmente ambas tareas bajo el mismo paradigma generativo, con sistemas como ncNet que generan especificaciones de visualización directamente y NL4DV-LLM que emplea GPT-4 para determinar tipos de gráfico y generar código de visualización.

Séptimo, mecanismos de seguridad integrables. PICARD demuestra que la decodificación restringida puede verificar la validez gramatical SQL en tiempo real durante la generación, reduciendo los errores de ejecución del 12 % al 2 %. Esta técnica es extensible para restringir la generación a operaciones de solo lectura (prohibiendo INSERT, UPDATE, DELETE, DROP), un requisito crítico para el sistema propuesto.

Arquitectura Recomendada para el Proyecto

Para el stack de código abierto propuesto, la configuración óptima consiste en: CodeLlama-13B o LLaMA-3-8B ajustado con QLoRA sobre datos Spider aumentados con ejemplos en español; RAG para schema linking dinámico embebiendo descripciones de tablas y columnas en un almacén vectorial como ChromaDB o FAISS; decodificación restringida estilo PICARD para garantizar SQL válido y seguro (solo SELECT); y un módulo de visualización automática que analiza la estructura de los resultados —tipos de datos, cardinalidades, presencia de agregaciones— para seleccionar el tipo de gráfico apropiado siguiendo la taxonomía de NL4DV. Para el prototipado inicial, T5-Base (220 M parámetros) con PICARD ofrece el 80.2 % EX en Spider con apenas 0.5 GB de VRAM para inferencia, constituyendo un punto de partida accesible.

Conclusión

El análisis comparativo de seis familias de técnicas de inteligencia artificial aplicadas al problema Text-to-SQL revela una jerarquía clara en la que los Transformers y LLMs dominan por márgenes empíricos insuperables. La evolución del campo se articula en tres eras bien diferenciadas: la era de los sistemas basados en reglas (décadas de 1970 a 2010), que ofrecían precisión determinista pero cobertura limitada; la era neural con RNN y GNN (2017–2021), que introdujo el aprendizaje automático pero mostró limitaciones ante la complejidad del SQL; y la era LLM (2022–presente), que ha acercado la precisión automática al rendimiento humano en benchmarks controlados.

Un hallazgo crítico es que la brecha entre los benchmarks académicos y los escenarios empresariales reales permanece sustancial: Spider 2.0 muestra que incluso los mejores modelos colapsan al 21 % en tareas de producción, frente al 91 % en Spider 1.0. Esto sugiere que las métricas actuales sobreestiman la madurez del campo y que la integración de mecanismos complementarios —RAG para el manejo de esquemas extensos, decodificación restringida para seguridad, validación por ejecución y retroalimentación interactiva con los usuarios— no es opcional sino imprescindible para despliegues en producción. Para el proyecto descrito, la combinación de un LLM de código abierto ajustado (CodeLlama-13B o LLaMA-3-8B con QLoRA), RAG para schema linking, PICARD para seguridad SQL y un módulo de visualización basado en correspondencia tipo-gráfico constituye la arquitectura con mayor probabilidad de éxito dado el estado actual del arte.

Referencias Bibliográficas

- Androutsopoulos, I., Ritchie, G. D., & Thanisch, P. (1995). Natural language interfaces to databases: An introduction. *Natural Language Engineering*, 1(1), 29–81. Cambridge University Press.
- Bohacek, M. & Bravansky, M. (2024). When XGBoost outperforms GPT-4 on text classification: A case study. *Proceedings of the 4th Workshop on Trustworthy NLP (TrustNLP 2024)*, pp. 51–60. ACL.
- Bogin, B., Berant, J., & Gardner, M. (2019). Representing schema structure with graph neural networks for text-to-SQL parsing. *Proceedings of ACL 2019*, pp. 4560–4565.
- Cao, R., Chen, L., Chen, Z., Zhao, Y., Zhu, S., & Yu, K. (2021). LGESQL: Line graph enhanced text-to-SQL model with mixed local and non-local relations. *Proceedings of ACL-IJCNLP 2021*, Vol. 1, pp. 2541–2555.
- Chen, T. & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of KDD 2016*, pp. 785–794. ACM.
- Dong, X., Zhang, C., Ge, Y., Mao, Y., Gao, Y., Chen, L., Lin, J., & Lou, D. (2023). C3: Zero-shot text-to-SQL with ChatGPT. *arXiv preprint arXiv:2307.07306*.
- Dou, L., Gao, Y., Pan, M., Wang, D., Che, W., Zhan, D., & Lou, J.-G. (2023). MultiSpider: Towards benchmarking multilingual text-to-SQL semantic parsing. *AAAI Conference on Artificial Intelligence*, 37, pp. 12745–12753.
- Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., & Zhou, J. (2024). Text-to-SQL empowered by large language models: A benchmark evaluation. *Proceedings of the VLDB Endowment*, 17(5), 1132–1145.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems (NeurIPS 2017)*.
- Lei, F., Chen, J., Ye, Y., Cao, R., Shin, D., Su, H., ... & Yu, T. (2024). Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. *arXiv preprint arXiv:2411.07763*. ICLR 2025 Oral.
- Li, F. & Jagadish, H. V. (2014). NaLIR: An interactive natural language interface for querying relational databases. *Proceedings of ACM SIGMOD 2014*.
- Li, H., Zhang, J., Li, C., & Chen, H. (2023a). RESDSQL: Decoupling schema linking and skeleton parsing for text-to-SQL. *Proceedings of the 37th AAAI Conference on AI*, 37, pp. 13067–13075.
- Li, H., Zhang, J., Li, C., & Chen, H. (2024b). CodeS: Towards building open-source language models for text-to-SQL. *Proceedings of ACM SIGMOD 2024*.
- Li, J., Hui, B., Qu, G., Yang, J., Li, B., et al. (2024a). Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs. *Advances in Neural Information Processing Systems (NeurIPS)*, 36.
- Li, J., Hui, B., Qu, G., Yang, J., Li, B., et al. (2023b). Graphix-T5: Mixing pre-trained transformers with graph-aware layers for text-to-SQL parsing. *Proceedings of the 37th AAAI Conference on AI*.

- Luo, Y., Tang, N., Li, G., Chai, C., Li, W., & Qin, X. (2021). Synthesizing natural language to visualization (NL2VIS) benchmarks from NL2SQL benchmarks. *Proceedings of ACM SIGMOD 2021*, pp. 1235–1247.
- Pourreza, M. & Rafiei, D. (2023). DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. *Advances in Neural Information Processing Systems (NeurIPS 2023)*.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 1–67.
- Scholak, T., Schucher, N., & Bahdanau, D. (2021). PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. *Proceedings of EMNLP 2021*, pp. 9895–9901.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., ... & Scialom, T. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Vapnik, V. (1995). *The Nature of Statistical Learning Theory*. Springer-Verlag.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS 2017)*, 30.
- Wahba, Y., Madhavji, N., & Steinbacher, J. (2022). A comparison of SVM against pre-trained language models (PLMs) for text classification tasks. *arXiv preprint arXiv:2211.02563*.
- Wang, B., Shin, R., Liu, X., Polozov, O., & Richardson, M. (2020). RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers. *Proceedings of ACL 2020*, pp. 8372–8388.
- Woods, W. A., Kaplan, R. M., & Nash-Webber, B. (1972). *The Lunar Sciences Natural Language Information System: Final Report*. BBN Report 2378, Bolt Beranek and Newman Inc.
- Xu, X., Liu, C., & Song, D. (2017). SQLNet: Generating structured queries from natural language without reinforcement learning. *arXiv preprint arXiv:1711.04436*.
- Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., ... & Radev, D. (2018). Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. *Proceedings of EMNLP 2018*, pp. 3911–3921.
- Zhong, R., Yu, T., & Klein, D. (2020). Semantic evaluation for text-to-SQL with distilled test suites. *Proceedings of EMNLP 2020*.
- Zhong, V., Xiong, C., & Socher, R. (2017). Seq2SQL: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103*.